SplitSmart

An AI-Assisted Group Expense Sharing, Trip Fund & Debt Minimization Engine

Project Assignment Submission Details
• Course: Python Programming
• Group info: Team(2 Members)

Member 1 :
- Name: Preeti Thakur
- Roll Number: 24EEB0A49
- Department & Class: EEEA

Member 2 :
- Name: Mitakshi
- Roll Number: 24EEB0B37
- Department & Class: EEEB

1. Project Overview & Problem Statement

Managing shared expenses during vacations, flat-sharing, and group outings is notoriously prone to confusion and manual calculation errors. When multiple members pay for different expenses at different times, settling up manually leads to tangled webs of circular debts, rounding discrepancies on itemized bills, and awkward accounting for families traveling with dependent children.

This project, SplitSmart, solves this with a centralized web application combining zero-drift financial mathematics with modern AI automation. It incorporates a greedy bipartite debt-minimization algorithm that compresses complex debt graphs to the theoretical minimum number of transactions (at most N - 1 payments), automatically rolls dependent debts onto parent sponsors, supports pre-funded group kitties, and provides fast, hands-free expense entry via Google Gemini natural text parsing and receipt photo scanning.

2. Technology Stack & System Architecture

The application is structured into decoupled frontend, backend, database, and algorithmic layers:

Layer	Technologies Used	Architectural Role & Key Function
Frontend / UI	HTML5, CSS3, JavaScript, Bootstrap 5	Responsive dashboards, dynamic modal forms, live debt balance cards, and interactive dish tagging.
Backend Framework	Python 3.10+, Flask 3.0	RESTful routing, Blueprint modularization, session-based authentication, and request lifecycle management.
Database Layer	MySQL 8.0+, mysql-connector-python	ACID-compliant relational storage, foreign key

		cascades, and multi-threaded connection pooling.
Mathematical Engines	Python standard library (decimal.Decimal)	Pure decimal financial arithmetic with ROUND_HALF_UP rounding and zero-drift penny reconciliation.
Artificial Intelligence	Google GenAI SDK (Gemini Flash), difflib	Natural language text prompt extraction, multimodal receipt OCR parsing, and fuzzy member name matching.

3. Uploaded Project Files & Roles

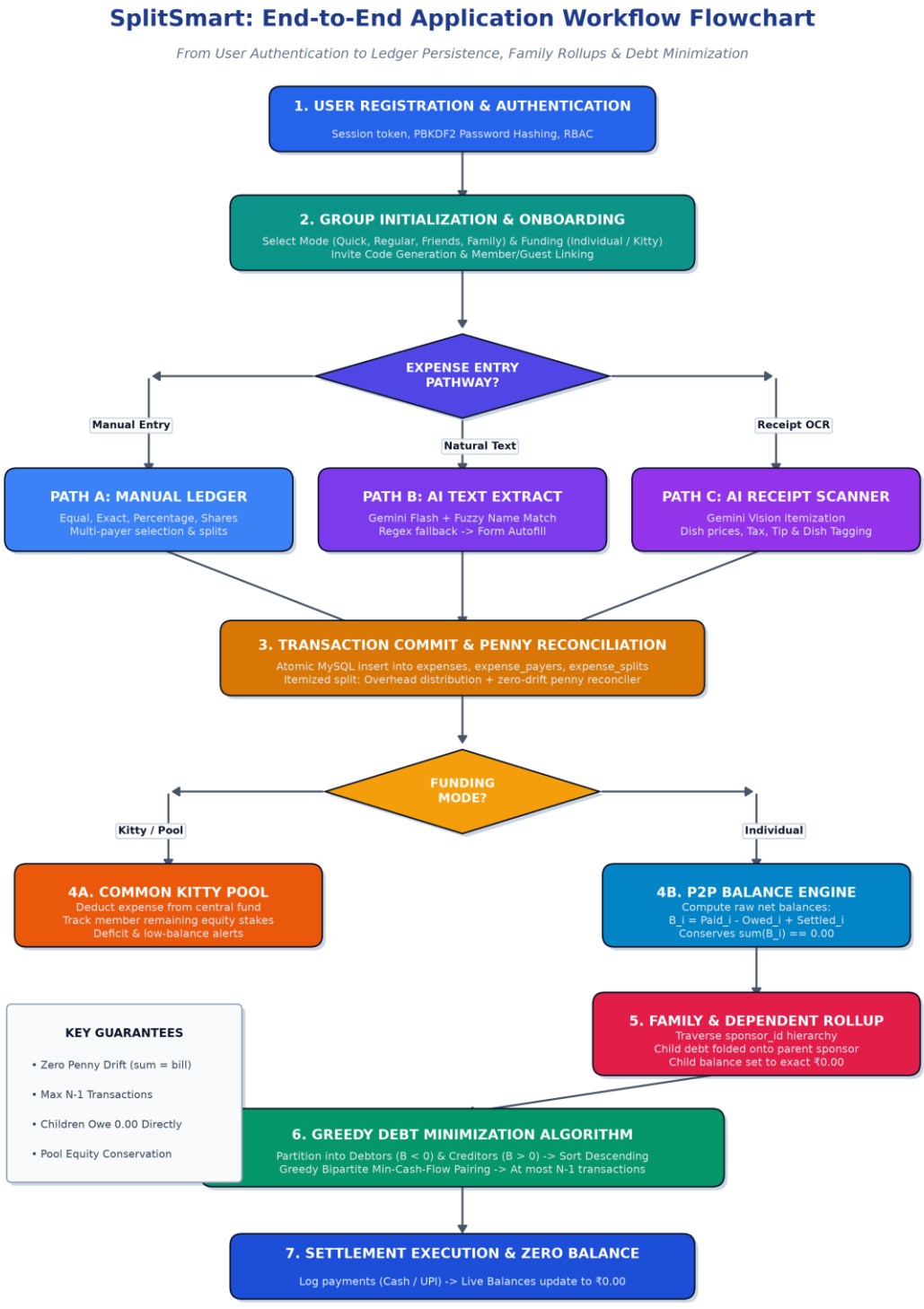
Below is the complete summary of all files included in this project submission:

Filename	File Type	Description & Role in Project
`run.py`	Python Script	Application launcher: starts the Flask web server (port 5000) with environment configurations.
`schema.sql`	Database DDL	Complete MySQL schema defining users, groups, members, expenses, payers, splits, settlements, and kitty fund tables.
`db.py`	Python Module	Database connection pool manager implementing MySQL connection pooling, cursor contexts, and query runners.
`app/__init__.py`	Flask App Factory	Initializes the Flask application, configures session management, and registers all feature Blueprints.
`app/config.py`	Config Script	Central configuration reading MySQL credentials, SECRET_KEY, and GEMINI_API_KEY from environment.
`app/routes/expenses.py`	Flask Blueprint	Expense ledger controller: handles expense CRUD across Equal, Exact, %, Shares, and Itemized split modes.
`app/routes/groups.py`	Flask Blueprint	Group management controller: group creation (4 modes), invite code generation, and dashboard views.
`app/routes/settlements.py`	Flask Blueprint	Settlement controller: computes live balances, displays simplified settlement plans, and logs repayments.
`app/routes/fund.py`	Flask Blueprint	Common trip fund controller: records upfront member deposits, tracks pot balance, and computes member equity.
`app/routes/ai.py`	Flask Blueprint	AI controller: exposes endpoints for natural language text parsing and multimodal receipt OCR

		scanning.
services/debt_simplifier.py	Python Service	Core debt minimization algorithm: sorts net debtors and creditors to compute minimal bipartite settlements.
services/balance_engine.py	Python Service	Mathematical balance engine: computes raw net balances, family sponsor rollups, and pool equity stakes.
services/expense_service.py	Python Service	Transactional expense engine: multi-payer validation, split mode calculations, and atomic MySQL inserts.
services/receipt_math.py	Python Service	Itemized receipt math engine: distributes taxes/tips proportionally and reconciles penny-rounding drift.
services/ai_nl_parser.py	Python Service	Natural language parser: Gemini Flash LLM prompt extraction with fuzzy member name matching and regex fallback.
services/ai_receipt_parser.py	Python Service	Multimodal receipt vision parser: Gemini Vision OCR line-item and total extraction from image/PDF receipts.
templates/groups/dashboard.html	HTML Template	Primary group dashboard displaying net balances, simplified settlement plan, expense list, and kitty status.
static/js/main.js & nl_modal.js	JavaScript Files	Client-side controllers handling AJAX requests, modal form population, and AI extract triggers.
requirements.txt	Config File	Lists necessary Python libraries (Flask, mysql-connector-python, google-genai, Pillow, matplotlib) for one-click setup.
flowchart.png	Image Asset	System architecture flowchart illustrating end-to-end data ingestion, ledger persistence, and settlement pipeline.
tests/test_math_engines.py	Unit Test Suite	Automated test suite verifying zero-drift decimal math, penny reconciliation, sponsor rollups, and debt minimization.

4. System Flowchart & Operational Pipeline

The complete end-to-end operational workflow of the SplitSmart application is illustrated in the flowchart below:



Flowchart Diagram: SplitSmart End-to-End Application Workflow

Brief Workflow Summary:

- **1. Group Setup:** Users register, create/join groups with unique invite codes, and select funding modes (Individual vs Kitty Pool).
- **2. Expense Ingestion:** Expenses enter via manual forms (Equal, Exact, %, Shares, Itemized), AI text parsing, or receipt photo OCR.
- **3. Ledger Persistence:** Records commit atomically to MySQL with proportional tax/tip math and zero-drift penny reconciliation.
- **4. Balance Routing & Rollups:** Kitty pool expenses deduct from the pot; individual expenses compute net balances, folding child debts onto parent sponsors.
- **5. Debt Settlement:** Greedy bipartite matching minimizes mutual debts into at most N - 1 direct payments settled via Cash or UPI.

5. How to Run the Program (Step-by-Step Guide)

The project is completely self-contained. The database schema (schema.sql), full Flask backend, and automated test suites are included and ready to run immediately on any Windows, macOS, or Linux PC.

Prerequisites & Environment Setup

• Python: Python 3.10 or higher installed on your PC.
• MySQL Server: MySQL 8.0+ or MariaDB (via XAMPP / WampServer) running locally.
• Virtual Environment Note: Setting up a virtual environment (venv) is standard practice to isolate project libraries, but running directly in your existing Python environment via pip install -r requirements.txt is also fully supported.
• Environment Variables (.env): Database credentials and secret keys are stored in .env in the project root.

Step 1: Open the Project Directory in Terminal

Extract the submission zip file to any folder on your computer. Open terminal/command prompt and navigate into the folder:

```
cd path/to/SplitSmart
```

Step 2: (Optional but Recommended) Setup a Virtual Environment

To isolate dependencies, create and activate a clean virtual environment:

```
# Windows:
python -m venv venv
venv\Scripts\activate

# macOS / Linux:
python3 -m venv venv
source venv/bin/activate
```

Step 3: Database Setup (Create Database & Import Schema)

In your MySQL client (phpMyAdmin, Workbench, or command line), create the database and import schema.sql:

```
CREATE DATABASE splitsmart_db CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;

# Import schema via terminal (or use phpMyAdmin 'Import' tab):
mysql -u root -p splitsmart_db < schema.sql
```

Step 4: Configure Database Credentials (.env)

Ensure the .env file in the root folder matches your local MySQL setup (default root with blank password works for XAMPP):

```
DB_HOST=127.0.0.1
DB_PORT=3306
DB_USER=root
DB_PASSWORD=
DB_NAME=splitsmart_db
SECRET_KEY=splitsmart-dev-secret-key-123
GEMINI_API_KEY=   # Optional: Add key, or leave blank to use automatic offline
fallbacks
```

Step 5: Install Required Dependencies

Run the following command to install the required Python libraries:

```
pip install -r requirements.txt
```

Step 6: (Optional) Run Automated Mathematical Test Suites

Verify that zero-drift decimal arithmetic, debt minimization, and sponsor rollups pass all unit tests:

```
python -m unittest tests/test_math_engines.py
# Output: Ran 4 tests ... OK
```

Step 7: Launch the Web Application

Start the interactive Flask web server with:

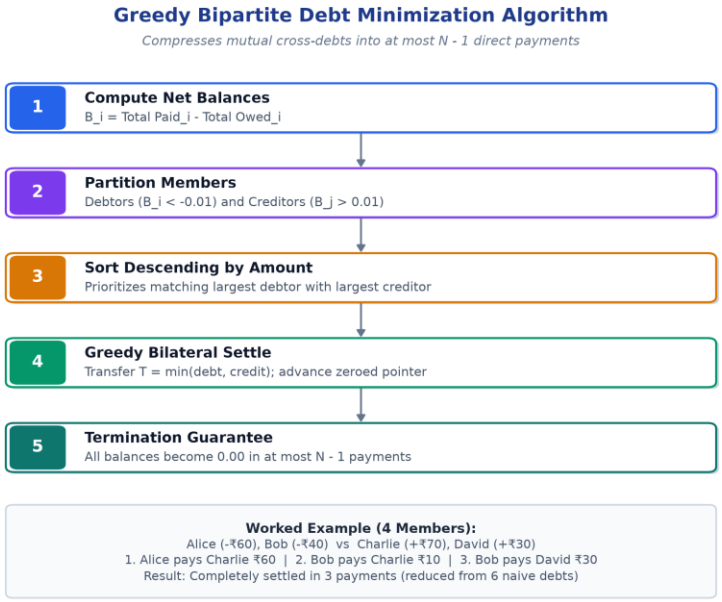
```
python run.py
```

The server will start at: http://127.0.0.1:5000. Open your browser and navigate to the link to register, create groups, and start managing expenses.

6. Core Mathematical Logic & Algorithms

6.1 Greedy Bipartite Debt Minimization

Computes each member's net balance (Total Paid - Total Owed) and greedily matches the largest debtor with the largest creditor. This settles at least one participant per step, provably compressing complex mutual debts into at most N - 1 direct payments.



Flowchart: Greedy Bipartite Debt Minimization Steps

6.2 Family Unit & Dependent Rollup Engine

Uses a cycle-safe sponsor_id hierarchy to automatically roll child debts onto their designated parent sponsor. Dependents always maintain an exact ₹0.00 settlement balance, allowing parents to settle a single consolidated net amount.

6.3 Proportional Itemized Split Engine & Penny Reconciliation

Divides individual dishes among tagged eaters and distributes taxes, tips, and discounts proportionally based on dish subtotals. An automated zero-drift reconciler assigns any residual rounding penny to the top spender, guaranteeing 100% exact bill conservation.

6.4 Common Kitty Pool (Trip Fund) Accounting

Manages collective upfront trip funds by deducting group expenses directly from the pot. Member equity stakes (Contribution - Consumption) are continuously tracked, triggering instant deficit alerts if any member overspends.

7. Testing & Verification

The project includes automated unit and integration test suites in the tests/ folder. These tests verify decimal arithmetic precision, ensure the penny reconciler prevents rounding drift, confirm that children's balances

roll up to parents leaving children at 0.00, and assert that the debt minimizer provably settles all mutual balances in the fewest possible transactions.

8. Visual Results & Application Screenshots

This section presents the primary user interfaces and workflow stages of the SplitSmart application, including user authentication, group balance tracking, multi-mode expense creation, AI text parsing, and trip fund pool monitoring.

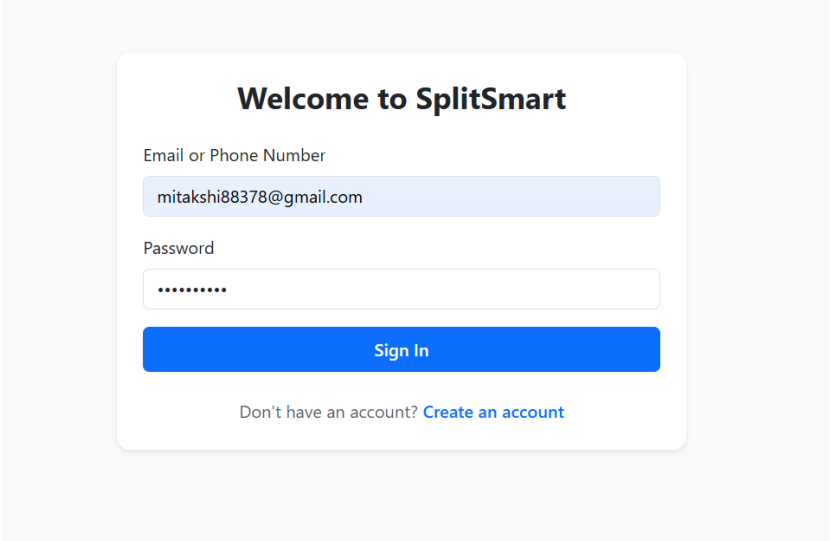


Figure 1: User Login and Registration Interface

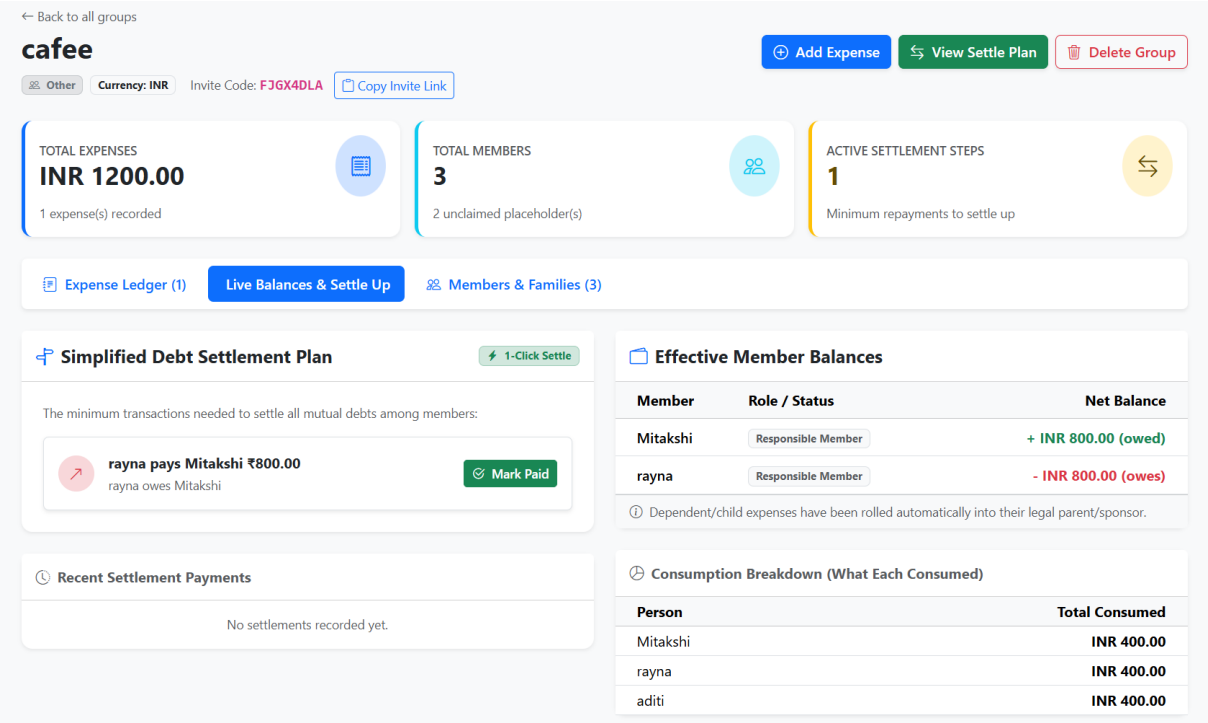


Figure 2: Group Dashboard with Member Balances & Simplified Settlement Plan

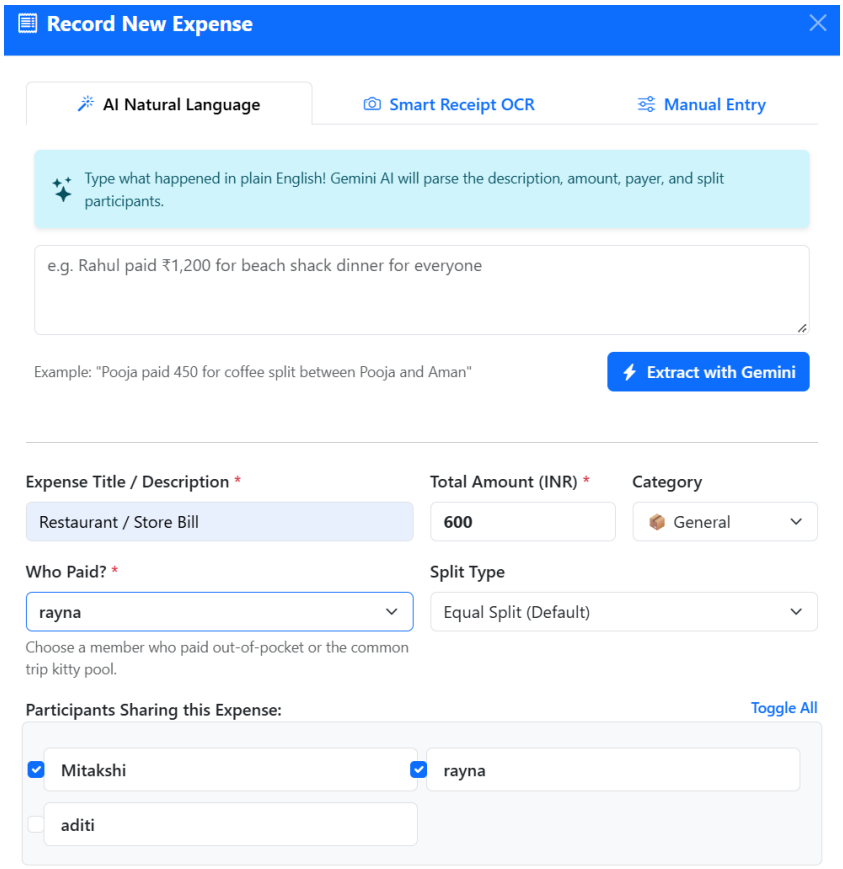


Figure 3: Expense Addition Modal (Equal, Exact, and Itemized Splits)

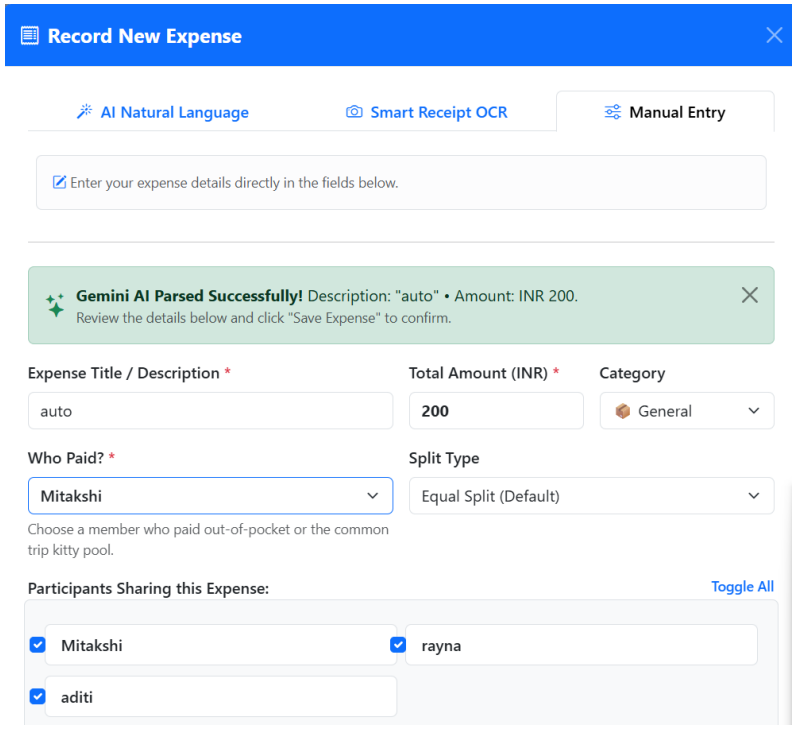


Figure 4: AI Natural Language Expense Parser in Action

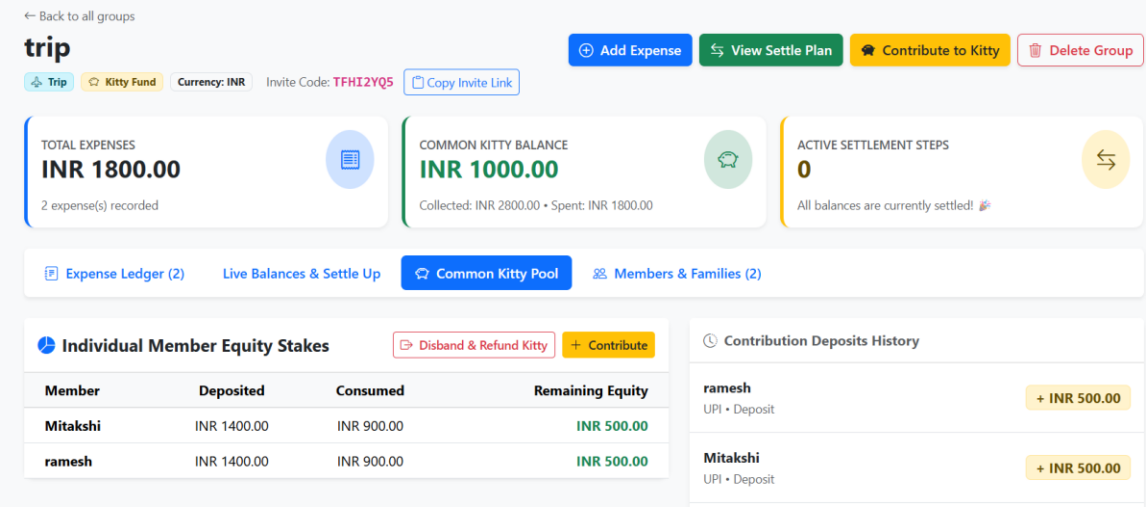


Figure 5: Common Kitty (Trip Fund) Pool Status & Equity Breakdown

9. Conclusion

The SplitSmart application provides a practical, mathematically verified, and AI-enhanced approach to group expense management. By replacing complex circular debts with an optimal greedy bipartite minimization algorithm, providing seamless family dependent rollups, guaranteeing zero-drift penny reconciliation, and automating data entry via Google Gemini AI, the system gives users complete financial transparency and confidence when settling up shared expenses.